C++ Midway Project

Video Game Library Management System

1. Project Overview

For my C++ midway project, I created a Video Game Library Management System. The program allows users to add video games, view existing records, search for games using their IDs, delete games, and save or load records using a text file.

The program uses a menu system so users can select different actions. Each game has a unique ID, a title, and a rating out of 10.

2. C++ Concepts Used

Data Types and Sizes: The program uses integers for game IDs, doubles for ratings, and strings for game titles. The `sizeof()` operator displays the memory sizes of different data types.

File Types: The program uses a `.txt` file named `games.txt` to store game records.

Pointers: A pointer is used to access and display a game found through the search function.

Arrays: An array stores up to 100 game objects in the library.

Binary Search: The program uses binary search to locate a game by its unique ID. Records are kept sorted by ID so the search works correctly.

Strings: Strings store video game titles, including titles containing spaces.

File Input/Output: The program uses `ofstream` to save records and `ifstream` to load records from `games.txt`.

Classes: The program contains two classes. The `Game` class stores individual game information, while the `GameLibrary` class manages the collection and its functions.

3. Program Testing and Results

Test 1 — Viewing Games

The program initially displayed three games: Dark Souls, Disco Elysium, and Elden Ring, with their IDs and ratings.

```
Game added successfully!
Game added successfully!
Game added successfully!

========== GAME LIBRARY ==========
1. Add a game
2. View all games
3. Search for a game
4. Delete a game
5. Save games to file
6. Load games from file
7. Show data type sizes
8. Exit
Enter your choice: 2

=== ALL GAMES ===
ID: 101 | Title: Dark Souls | Rating: 9/10
ID: 102 | Title: Disco Elysium | Rating: 9.5/10
ID: 103 | Title: Elden Ring | Rating: 9.2/10
```

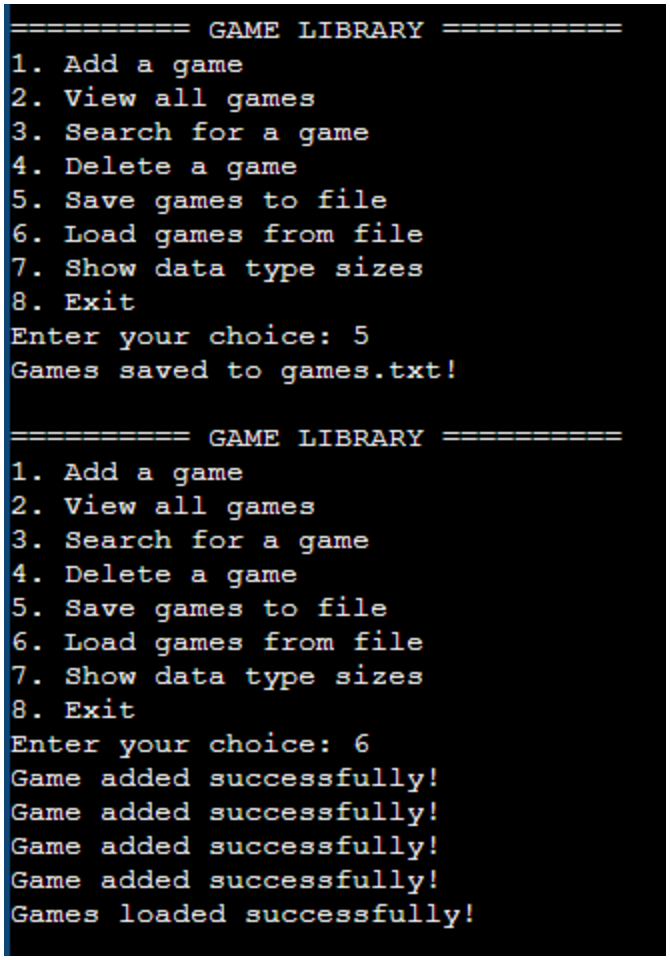
Test 2 — Adding a Game

I selected option 1 and entered a new game with ID 104 and a rating of 9.4. The program displayed "Game added successfully!" and the record appeared when viewing the library.

```
========== GAME LIBRARY ==========
1. Add a game
2. View all games
3. Search for a game
4. Delete a game
5. Save games to file
6. Load games from file
7. Show data type sizes
8. Exit
Enter your choice: 1
Enter game ID: 104
Enter game title: Hollow Knight
Enter rating (0-10): 9.4
Game added successfully!
```

Test 3 — Saving and Loading

I selected option 5 to save the library to `games.txt`. I then selected option 6 to load the records. The program confirmed that the games were successfully loaded.



Test 4 — Deleting a Game

I selected option 4 and deleted the game with ID 101. After viewing the library again, Dark Souls was no longer listed, confirming that the deletion worked.

```
========== GAME LIBRARY ==========
1. Add a game
2. View all games
3. Search for a game
4. Delete a game
5. Save games to file
6. Load games from file
7. Show data type sizes
8. Exit
Enter your choice: 4
Enter game ID to delete: 101
Game deleted successfully!

========== GAME LIBRARY ==========
1. Add a game
2. View all games
3. Search for a game
4. Delete a game
5. Save games to file
6. Load games from file
7. Show data type sizes
8. Exit
Enter your choice: 2

=== ALL GAMES ===
ID: 102 | Title: Disco Elysium | Rating: 9.5/10
ID: 103 | Title: Elden Ring | Rating: 9.2/10
ID: 104 | Title: Hollow Knight | Rating: 9.4/10
```

Test 5 — Data Type Sizes

I selected option 7 to display the memory sizes of different data types. The program displayed:

- int: 4 bytes
- double: 8 bytes
- string: 32 bytes
- Game object: 48 bytes
- Game pointer: 8 bytes

These sizes reflect the compiler environment used to run the program.

```
========== GAME LIBRARY ==========
1. Add a game
2. View all games
3. Search for a game
4. Delete a game
5. Save games to file
6. Load games from file
7. Show data type sizes
8. Exit
Enter your choice: 7

=== DATA TYPE SIZES ===
int: 4 bytes
double: 8 bytes
string: 32 bytes
Game object: 48 bytes
Game pointer: 8 bytes
```

Test 6 — Binary Search

```
========== GAME LIBRARY ==========
1. Add a game
2. View all games
3. Search for a game
4. Delete a game
5. Save games to file
6. Load games from file
7. Show data type sizes
8. Exit
Enter your choice: 3
Enter game ID to search: 103
Game found!
ID: 103 | Title: Elden Ring | Rating: 9.2/10
```

4. Conclusion

This project helped me practice combining several C++ concepts into one working application. I used classes to organize the program, arrays to store records, binary search to find games, and file input/output to save and load information.

The project also helped me understand how different functions work together in a larger program and how to test whether records are being added, deleted, and stored correctly.